PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

Gustavo Henrique da Silva

PROBLEMAS CLÁSSICOS DE CONCORRÊNCIA

Relatório

2026

1. Introdução

Os problemas clássicos de sincronização são modelos fundamentais em sistemas concorrentes, já que representam cenários em que múltiplas tarefas competem por recursos compartilhados. Nesse contexto, a coordenação correta entre threads ou goroutines é essencial a fim de evitar inconsistências, bloqueios e desperdício de tempo de execução.

Neste trabalho foram implementadas soluções na linguagem de programação Go para dois problemas clássicos de concorrência: o Jantar dos Filósofos e o problema do Produtor/Consumidor com buffer limitado. Ambos envolvem acesso simultâneo a recursos compartilhados e exigem a utilização de mecanismos de sincronização, tais como channels, select, wait groups e semáforos.

O objetivo principal foi analisar como cada solução se comporta sob concorrência, identificando situações de deadlock, starvation e fairness. Além disso, foi necessário verificar o encerramento ordenado dos processos e coletar dados de execução para analisar o comportamento dos algoritmos em diferentes condições.

2. Jantar dos Filósofos

O problema do Jantar dos Filósofos foi proposto como um modelo clássico de concorrência em que vários filósofos compartilham um conjunto de garfos posicionados entre eles. Cada filósofo precisa de dois garfos para comer. O desafio é garantir que os filósofos consigam realizar refeições sem entrar em deadlock, ou seja, sem ficarem todos esperando por recursos que nunca serão disponibilizados.

A solução implementada em Go modela cada filósofo como uma goroutine e cada garfo como um channel com capacidade unitária. Essa representação permite que a disponibilidade de cada recurso seja controlada por sincronização explícita.

2.1 Versão base com deadlock

A versão base foi desenvolvida seguindo a regra de que todos os filósofos pegam o primeiro garfo da esquerda antes de tentar pegar o garfo da direita. Apesar de simples de se implementar, a regra cria um cenário de espera circular. Se todos filósofos seguram o garfo da esquerda ao mesmo tempo, cada um passa a esperar o garfo da direita, que está sendo segurado por outro filósofo. Este é o ponto onde ocorre o deadlock.

Essa situação ocorre por haver sincronização entre os processos, mas não há mecanismos para quebra da circularidade de espera. O algoritmo fica preso em um estado em que nenhum filósofo consegue avançar, mesmo que todos estejam ativos e prontos para continuar.

2.2 Estratégias para eliminar o deadlock

Para que o deadlock seja eliminado, foram adotadas estratégias que quebram a simetria do sistema ou limitam o acesso simultâneo aos recursos. A primeira estratégia foi a quebra de simetria por hierarquia de recursos. Nesse caso, a ordem de aquisição dos garfos foi alterada para alguns filósofos, de modo que o último filósofo pega primeiro o garfo da direita e só depois o da esquerda. Desta forma, o sistema deixa de permitir a espera circular geral, pois nem todos os filósofos seguem o mesmo padrão de aquisição de recursos.

A segunda estratégia foi a limitação da quantidade de filósofos simultaneamente na mesa ou a introdução de um mecanismo de arbitragem centralizada, como a presença de um garçom que autoriza a entrada de filósofos. Essa abordagem reduz o número de filósofos competindo por garfos simultaneamente e evita que bloqueiem uns aos outros.

Essas estratégias funcionam por reduzirem a chance de todos os filósofos entrarem em um estado de retenção e espera simultânea. Assim, a circularidade que gera o deadlock deixa de ocorrer.

2.3 Análise de deadlock e starvation

A versão base apresenta deadlock. Isso pode ser explicado pelas condições de Coffman: exclusão mútua, retenção e espera, ausência de preempção e espera circular. Todos os filósofos

mantêm um garfo em posse e esperam o outro, formando um ciclo em que nenhum consegue progredir.

A versão segura não apresenta deadlock, já que a ordem de aquisição dos recursos deixa de ser simétrica. Assim, a espera circular deixa de ocorrer e cada filósofo consegue eventualmente liberar os garfos após comer.

Em relação à starvation, a versão base também pode ser considerada problemática, pois, em um cenário de deadlock, alguns filósofos podem ficar indefinidamente sem conseguir comer. Já nas versões com quebra de simetria ou limitação de acesso, starvation não é esperada de forma persistente, desde que haja equilíbrio na execução e que os filósofos recebam acesso aos recursos de maneira justa.

A starvation ocorre quando um processo fica impedido de executar por tempo excessivo, mesmo sem deadlock. No caso do jantar dos filósofos, isso geralmente ocorre quando algum filósofo é sistematicamente atrasado na fila de acesso aos recursos. Nas estratégias adotadas, a ordem de aquisição dos garfos e a limitação do número de filósofos na mesa reduzem esse problema, tornando o acesso mais equilibrado.

2.4 Fairness e dados de execução

A fairness de cada estratégia foi analisada por meio do número de refeições realizadas por filósofo e do tempo médio de espera para obter os garfos. Em uma execução correta, todos filósofos devem receber acesso razoável aos recursos, evitando que alguns consumam muito mais do que outros ou esperem mais tempo de forma injusta.

Os dados coletados mostraram que a versão base, por apresentar deadlock, não oferece fairness, pois os filósofos ficam bloqueados antes de concluir as refeições. Já as versões seguras apresentaram comportamento mais equilibrado, com tempos de espera menores e distribuição mais homogênea entre os filósofos. Isso sugere que a quebra de simetria e a limitação de acesso melhoram tanto a correção quanto a justiça do sistema.

Cada estratégia segura foi executada três vezes, com cinco filósofos e cem refeições por filósofo. Todas as execuções seguras concluíram as 500 refeições previstas. A estratégia de hierarquia apresentou menor duração e menor espera média geral, mas maior variação entre os filósofos. A limitação a N-1 filósofos apresentou esperas médias mais próximas entre si, embora tenha demorado mais. Esses valores descrevem as execuções realizadas e podem variar entre execuções. Na versão base, o deadlock foi detectado antes da conclusão de qualquer refeição.

Estratégia segura	Execuções concluídas	Duração média	Espera média geral	Menor - maior espera média entre filósofos
Hierarquia de garfos	3 de 3 (500 refeições cada)	5,668s	30,664ms	20,639 - 36,311ms
Limite de N-1 filósofos	3 de 3 (500 refeições cada)	9,976s	79,158ms	78,879 - 79,419ms

A tabela abaixo detalha a espera média de cada filósofo, calculada a partir das três execuções:

Filósofo	Hierarquia de garfos	Limite de N-1 filósofos
0	36,107ms	78,879ms

1	32,129ms	79,144ms
2	28,134ms	79,140ms
3	20,639ms	79,419ms
4	36,311ms	79,209ms

2.5 Encerramento ordenado

O encerramento ordenado foi implementado garantindo que todas as goroutines realizassem o número de iterações especificado e que os canais representando os garfos tivessem seus elementos liberados ao fim do processamento. O programa observa quando todas as refeições foram concluídas e encerra a execução sem deixar goroutines bloqueadas. Dessa forma, evita-se que o sistema fique preso aguardando recursos que já não serão liberados.

3. Produtor/Consumidor com Buffer Limitado

O problema do Produtor/Consumidor trata da interação entre dois tipos de processos: produtores, que geram itens; e consumidores, que os retiram de um buffer compartilhado. O buffer possui capacidade limitada, de modo que o produtor precisa bloquear quando o buffer está cheio, e o consumidor precisa bloquear quando o buffer está vazio.

Esse cenário é importante porque exige que dados compartilhados sejam acessados de forma segura e ordenada, evitando condições de corrida, perda de itens e inconsistências no estado do buffer.

3.1 Versão com channel

A primeira implementação utiliza um channel como buffer compartilhado. Esse modelo é simples e direto, dado que a própria linguagem Go já implementa o bloqueio de escrita e leitura quando o channel está cheio ou vazio. Assim, produtores e consumidores ficam sincronizados de forma nativa pela semântica do channel.

Essa abordagem reduz bastante a complexidade do código e ajuda a garantir que não ocorram duas escritas na mesma posição ou duas leituras do mesmo item. A utilização de select com timeout permite que, em alguns consumidores, a rotina continue funcionando mesmo quando não há itens disponíveis por algum tempo.

3.2 Versão com semáforos

A segunda implementação simula o buffer com uma estrutura compartilhada protegida por semáforos de contagem e exclusão mútua. Nesse caso, o buffer foi modelado como um vetor circular com índices de entrada e saída. O semáforo de itens disponíveis controla quando há dados para consumir, já o semáforo de espaço livre controla quando há espaço para produzir. A região crítica foi protegida por um mutex para garantir que dois produtores não escrevam na mesma posição e que dois consumidores não removam o mesmo item.

Essa abordagem reproduz o algoritmo clássico de sincronização do problema do produtor/consumidor, mostrando de forma explícita como os semáforos permitem controlar acesso concorrente ao buffer. Embora mais complexa que a versão com channel, ela enfatiza as regras de sincronização que garantem consistência do buffer

3.3 Encerramento ordenado

O encerramento ordenado foi implementado em ambas as versões para evitar deadlock ao final da execução. Os produtores terminam a geração de itens e, em seguida, os consumidores continuam drenando o buffer até consumir tudo o que ainda resta. Depois disso, o programa verifica se o total produzido é igual ao total consumido, confirmando que nenhum item foi perdido ou duplicado. Esse mecanismo é importante devido ao fato de que sem uma etapa de drenagem final, o

programa poderia terminar precocemente, antes de consumir os itens presentes no buffer, ou poderia bloquear consumidores esperando por dados que nunca mais seriam produzidos.

3.4 Consumo com timeout

No código, ao menos um consumidor foi implementado com select e time.After para contemplar um comportamento alternativo quando nenhum item chega em um intervalo de tempo definido. Essa escolha permite simular uma situação em que o consumidor continua sua execução mesmo que não receba dados, ao invés de ficar bloqueado indefinidamente. A relevância desse recurso se dá pela não dependência da concorrência apenas pela chegada de itens, mas também pela capacidade de reagir a tempos de espera e a eventos de timeout. O uso de select em conjunto com time.After demonstrou ser útil para testar a robustez do sistema.

3.5 Efeito da capacidade do buffer K

Para avaliar o efeito da capacidade do buffer, foram executados testes com duas goroutines produtoras, duas consumidoras e 200 itens. Cada combinação de implementação e capacidade foi executada uma vez com go run -race. A taxa de processamento foi calculada dividindo o total de itens consumidos pelo tempo de execução. A ocupação média foi estimada por amostras coletadas a cada milissegundo.

Implementação	Capacidade (K)	Tempo	Taxa de processamento	Ocupação média (itens)	Itens por consumidor (C0/C1)
Channel	1	1,065s	187,78 itens/s	0,00000	101 / 99
Channel	10	1,066s	187,68 itens/s	0,00000	100 / 100
Channel	100	1,049s	190,66 itens/s	0,00000	101 / 99
Semáforos	1	1,196s	167,20 itens/s	0,005484	99 / 101
Semáforos	10	1,201s	166,54 itens/s	0,006494	99 / 101
Semáforos	100	1,206s	165,85 itens/s	0,007986	100 / 100

Em todas as execuções, foram produzidos e consumidos os 200 itens. A distribuição entre os dois consumidores também foi próxima: cada um consumiu entre 99 e 101 itens.

Nesta configuração, aumentar K de 1 para 100 quase não alterou a taxa de processamento. A ocupação média registrada foi baixa em ambas implementações. Na versão com channel, o valor observado foi zero, indicando que as amostras não encontraram itens esperando no buffer, porém não há provas de que ele nunca conteve algum item entre uma amostra e outra. Como os consumidores retiravam os itens sem uma pausa artificial e os produtores produziam um item a cada 10ms, o buffer teve pouca oportunidade de acumulação de itens.

A versão com channel apresentou taxa de processamento maior nestas execuções, porém os resultados não permitem concluir que ela será sempre mais rápida. Foi realizada apenas uma execução por configuração e o desempenho depende das condições de execução. Para que o efeito de K seja melhor avaliado, é necessário a repetição de cada teste várias vezes para que seja calculada a média das execuções. Além disso, testar consumidores mais lentos poderia evidenciar com mais clareza de que forma um buffer maior permite que os produtores continuem trabalhando enquanto os itens aguardam para serem consumidos.

4. Conclusão

Os testes demonstraram como diferentes formas de sincronização afetam a execução de programas concorrentes. No jantar dos filósofos, a versão base confirmou o risco de deadlock: os filósofos ficaram esperando pelos garfos e não conseguiram concluir as refeições. Nas estratégias seguras, todos os filósofos concluíram as 100 refeições em cada execução. A hierarquia de garfos teve a menor duração média (cerca de 5,668 segundos) e a menor espera média geral, de 30,664ms. A estratégia que limita a N-1 o número de filósofos tentando pegar garfos demorou mais (cerca de 9,976 segundos) e teve espera média de 79,158ms. Contudo, as esperas entre os filósofos foram mais próximas nesta estratégia, sugerindo uma distribuição mais uniforme nas execuções realizadas.

No problema do Produtor/Consumidor, as duas implementações consumiram todos os 200 itens em cada teste, sem perda de dados. Com os parâmetros utilizados, variar a capacidade do buffer entre 1, 10 e 100 quase não alterou a taxa de processamento medida. A versão com channel apresentou taxas de processamento entre 187,68 e 190,66 itens por segundo. A versão com semáforos, entre 165,85 e 167,20 itens por segundo. A ocupação média observada também foi baixa, indicando que, nessa configuração, os consumidores retiraram os itens sem que eles fossem significativamente acumulados no buffer.

Esses resultados sugerem que a escolha da estratégia de sincronização influencia tanto o tempo de execução quanto a distribuição de espera. Porém, como cada configuração do Produtor/Consumidor foi testada apenas uma vez e os resultados dependem das condições de execução, não é possível afirmar que uma implementação será sempre mais rápida ou que determinada capacidade será sempre melhor. Em conjunto, os resultados evidenciam que é essencial prevenir deadlocks, confirmar que todos os itens foram processados e analisar o desempenho levando em conta as condições de cada teste.

5. Declaração do uso de IAs

Foram utilizados modelos LLM Chatgpt-6 e Claude Sonnet 5.5 para o auxílio na realização do presente trabalho, o primeiro tendo sido utilizado para revisão ortográfica do relatório e o segundo como auxílio na elaboração do trecho de código para extração dos dados de execução dos testes exibidos nas tabelas apresentadas no relatório.